

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №3

по дисциплине: "Логика и основы алгоритмизации в инженерных задачах"

на тему: "Динамические списки"

Выполнил:

Студент группы 24ВВВ3

Мингалёв М.А.

Принял:

к.т.н., доцент, Юрова О. В.

к.т.н., Деев М.В.

Пенза 2025

Цель

Изучение динамических списков.

Лабораторное задание

Задание:

а. Реализовать приоритетную очередь, путём добавления элемента в список в соответствии с приоритетом объекта (т.е. объект с большим приоритетом становится перед объектом с меньшим приоритетом).

б. На основе приведенного кода реализуйте структуру данных Очередь.

с. На основе приведенного кода реализуйте структуру данных Стек.

Результат программы

```
E:\gitnew\laba3\laba3\x64\Debug\laba3.exe
ПРОГРАММА ДЛЯ РАБОТЫ С СТРУКТУРАМИ ДАННЫХ
ПРИОРИТЕТНАЯ ОЧЕРЕДЬ:
  1. Добавить элемент
  2. Удалить элемент
  3. Просмотр
  4. Поиск по приоритету
ОЧЕРЕДЬ (FIFO):
  5. Добавить элемент
  6. Удалить элемент
  7. Просмотр
СТЕК (LIFO):
  8. Добавить элемент
  9. Удалить элемент
 10. Просмотр
  0. Выход
Выберите действие: 1
Введите название объекта: 10
Введите приоритет: 10
Элемент добавлен в приоритетную очередь

Выберите действие: 1
Введите название объекта: 10
Введите приоритет: 100
Элемент добавлен в приоритетную очередь

Выберите действие: 1
Введите название объекта: 100
Введите приоритет: -100
Элемент добавлен в приоритетную очередь

Выберите действие: 1
Введите название объекта: 1
Введите приоритет: 1
Элемент добавлен в приоритетную очередь
Выберите действие: 1
Введите название объекта: -1
Введите приоритет: -1
Элемент добавлен в приоритетную очередь
```

```
Выберите действие: 3

=== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ===
1. Число: 10, Приоритет: 100
2. Число: 10, Приоритет: 10
3. Число: 1, Приоритет: 1
4. Число: -1, Приоритет: -1
5. Число: 100, Приоритет: -100
```

Рисунок 1 – приоритетная очередь

```
E:\gitnew\laba3\laba3\x64\Debug\laba3.exe
ПРОГРАММА ДЛЯ РАБОТЫ С СТРУКТУРАМИ ДАННЫХ
ПРИОРИТЕТНАЯ ОЧЕРЕДЬ:
  1. Добавить элемент
  2. Удалить элемент
  3. Просмотр
  4. Поиск по приоритету
ОЧЕРЕДЬ (FIFO):
  5. Добавить элемент
  6. Удалить элемент
  7. Просмотр
СТЕК (LIFO):
  8. Добавить элемент
  9. Удалить элемент
  10. Просмотр
  0. Выход
Выберите действие: 5
Введите название объекта: 1
Элемент добавлен в очередь

Выберите действие: 5
Введите название объекта: 100
Элемент добавлен в очередь

Выберите действие: 5
Введите название объекта: 23
Элемент добавлен в очередь

Выберите действие: 5
Введите название объекта: -2
Элемент добавлен в очередь

Выберите действие: 5
Введите название объекта: -100
Элемент добавлен в очередь

Выберите действие: 7

=== ОЧЕРЕДЬ (FIFO) ===
1. Число: 1
2. Число: 100
3. Число: 23
4. Число: -2
5. Число: -100
```

Рисунок 2 – список «Очередь»

C:\ E:\gitnew\laba3\laba3\x64\Debug\laba3.exe

```
ПРОГРАММА ДЛЯ РАБОТЫ С СТРУКТУРАМИ ДАННЫХ
ПРИОРИТЕТНАЯ ОЧЕРЕДЬ:
  1. Добавить элемент
  2. Удалить элемент
  3. Просмотр
  4. Поиск по приоритету
ОЧЕРЕДЬ (FIFO):
  5. Добавить элемент
  6. Удалить элемент
  7. Просмотр
СТЕК (LIFO):
  8. Добавить элемент
  9. Удалить элемент
 10. Просмотр
  0. Выход
Выберите действие: 8
Введите название объекта: 100
Элемент добавлен в стек
Выберите действие: 8
Введите название объекта: 10
Элемент добавлен в стек
Выберите действие: 8
Введите название объекта: 2
Элемент добавлен в стек
Выберите действие: 8
Введите название объекта: 1
Элемент добавлен в стек
Выберите действие: 8
Введите название объекта: -1
Элемент добавлен в стек
Выберите действие: 8
Введите название объекта: 1000
Элемент добавлен в стек
Выберите действие: 10

=== СТЕК (LIFO) ===
1. Число: 1000
2. Число: -1
3. Число: 1
4. Число: 2
5. Число: 10
6. Число: 100
```

Рисунок 3 – список «Стек»

Листинг

Файл 1- laba3.c

```
#define _CRT_SECURE_NO_WARNINGS

#include <iostream>
#include <cstdlib>
#include <cstring>
#include <locale>

using namespace std;

struct node
{
    char inf[256];
    int priority;
    struct node* next;
};

struct node* priority_head = NULL, * priority_last = NULL;
struct node* queue_head = NULL, * queue_last = NULL;
struct node* stack_head = NULL, * stack_last = NULL;

struct node* get_struct(void);
struct node* get_struct_with_priority(void);

void priority_enqueue(void);
void priority_dequeue(void);
void view_priority_queue(void);
void search_by_priority(void);

void enqueue(void);
void dequeue(void);
void view_queue(void);

void push(void);
void pop(void);
void view_stack(void);

void clear_all_lists(void);
void show_menu(void);

struct node* get_struct(void)
{
    struct node* p = NULL;
    char s[256];

    p = (node*)malloc(sizeof(struct node));
    if (p == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        return NULL;
    }

    printf("Введите название объекта: ");
    scanf("%255s", s);
    if (*s == 0)
    {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }
    strcpy(p->inf, s);
    p->priority = 0;
    p->next = NULL;

    return p;
}
```

```

struct node* get_struct_with_priority(void)
{
    struct node* p = NULL;
    char s[256];
    int pr;

    p = (node*)malloc(sizeof(struct node));
    if (p == NULL)
    {
        printf("Ошибка при распределении памяти\n");
        return NULL;
    }

    printf("Введите название объекта: ");
    scanf("%255s", s);
    if (*s == 0)
    {
        printf("Запись не была произведена\n");
        free(p);
        return NULL;
    }

    printf("Введите приоритет: ");
    scanf("%d", &pr);

    strcpy(p->inf, s);
    p->priority = pr;
    p->next = NULL;

    return p;
}

// ===== ПОИСК ПО ПРИОРИТЕТУ =====
void search_by_priority(void)
{
    if (priority_head == NULL)
    {
        printf("Приоритетная очередь пуста\n");
        return;
    }

    int target_priority;
    printf("Введите приоритет для поиска: ");
    scanf("%d", &target_priority);

    struct node* current = priority_head;
    int found = 0;
    int count = 1;

    printf("\n=== РЕЗУЛЬТАТЫ ПОИСКА (приоритет %d) ===\n", target_priority);

    while (current != NULL)
    {
        if (current->priority == target_priority)
        {
            printf("%d. Число: %s, Приоритет: %d\n", count, current->inf, current->priority);
            found++;
            count++;
        }
        current = current->next;
    }

    if (!found)
    {
        printf("Элементы с приоритетом %d не найдены\n", target_priority);
    }
    else
    {
        printf("Найдено элементов: %d\n", found);
    }
}

```

```

}

// ===== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ =====

void priority_enqueue(void)
{
    struct node* p = NULL;
    struct node* current = NULL;
    struct node* prev = NULL;

    p = get_struct_with_priority();
    if (p == NULL)
        return;

    if (priority_head == NULL || p->priority > priority_head->priority)
    {
        p->next = priority_head;
        priority_head = p;
        if (priority_last == NULL)
            priority_last = p;
        printf("Элемент добавлен в приоритетную очередь\n");
        return;
    }

    current = priority_head;
    while (current != NULL && current->priority >= p->priority)
    {
        prev = current;
        current = current->next;
    }

    prev->next = p;
    p->next = current;

    if (current == NULL)
        priority_last = p;

    printf("Элемент добавлен в приоритетную очередь\n");
}

void priority_dequeue(void)
{
    if (priority_head == NULL)
    {
        printf("Приоритетная очередь пуста\n");
        return;
    }

    struct node* temp = priority_head;
    printf("Удален элемент: '%s' (приоритет: %d)\n", temp->inf, temp->priority);
    priority_head = priority_head->next;
    if (priority_head == NULL)
        priority_last = NULL;
    free(temp);
}

void view_priority_queue(void)
{
    struct node* struc = priority_head;
    if (priority_head == NULL)
    {
        printf("Приоритетная очередь пуста\n");
        return;
    }
    printf("\n===== ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ====\n");
    int count = 1;
    while (struc)
    {
        printf("%d. Число: %s, Приоритет: %d\n", count, struc->inf, struc->priority);
        struc = struc->next;
        count++;
    }
}

```

```

    }
}

// ===== ОБЫЧНАЯ ОЧЕРЕДЬ =====

void enqueue(void)
{
    struct node* p = NULL;
    p = get_struct();
    if (p == NULL)
        return;

    if (queue_head == NULL)
    {
        queue_head = p;
        queue_last = p;
    }
    else
    {
        queue_last->next = p;
        queue_last = p;
    }
    printf("Элемент добавлен в очередь\n");
}

void dequeue(void)
{
    if (queue_head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }

    struct node* temp = queue_head;
    printf("Удален элемент: '%s'\n", temp->inf);
    queue_head = queue_head->next;
    if (queue_head == NULL)
        queue_last = NULL;
    free(temp);
}

void view_queue(void)
{
    struct node* struc = queue_head;
    if (queue_head == NULL)
    {
        printf("Очередь пуста\n");
        return;
    }
    printf("\n=== ОЧЕРЕДЬ (FIFO) ===\n");
    int count = 1;
    while (struc)
    {
        printf("%d. Число: '%s'\n", count, struc->inf);
        struc = struc->next;
        count++;
    }
}

// ===== СТЕК =====

void push(void)
{
    struct node* p = NULL;
    p = get_struct();
    if (p == NULL)
        return;

    p->next = stack_head;
    stack_head = p;
    if (stack_last == NULL)

```



```

        stack_last = p;
        printf("Элемент добавлен в стек\n");
    }

void pop(void)
{
    if (stack_head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }

    struct node* temp = stack_head;
    printf("Удален элемент: '%s'\n", temp->inf);
    stack_head = stack_head->next;
    if (stack_head == NULL)
        stack_last = NULL;
    free(temp);
}

void view_stack(void)
{
    struct node* struc = stack_head;
    if (stack_head == NULL)
    {
        printf("Стек пуст\n");
        return;
    }
    printf("\n=== СТЕК (LIFO) ===\n");
    int count = 1;
    while (struc)
    {
        printf("%d. Число: %s\n", count, struc->inf);
        struc = struc->next;
        count++;
    }
}

void clear_all_lists(void)
{
    struct node* temp = NULL;
    while (priority_head != NULL)
    {
        temp = priority_head;
        priority_head = priority_head->next;
        free(temp);
    }
    priority_last = NULL;

    while (queue_head != NULL)
    {
        temp = queue_head;
        queue_head = queue_head->next;
        free(temp);
    }
    queue_last = NULL;

    while (stack_head != NULL)
    {
        temp = stack_head;
        stack_head = stack_head->next;
        free(temp);
    }
    stack_last = NULL;
}

void show_menu(void)
{
    printf(" ПРОГРАММА ДЛЯ РАБОТЫ С СТРУКТУРАМИ ДАННЫХ\n");
}

```

```

printf(" ПРИОРИТЕТНАЯ ОЧЕРЕДЬ: \n");
printf(" 1. Добавить элемент \n");
printf(" 2. Удалить элемент \n");
printf(" 3. Просмотр      \n");
printf(" 4. Поиск по приоритету\n");

printf(" ОЧЕРЕДЬ (FIFO):  \n");
printf(" 5. Добавить элемент \n");
printf(" 6. Удалить элемент \n");
printf(" 7. Просмотр      \n");

printf(" СТЕК (LIFO):      \n");
printf(" 8. Добавить элемент \n");
printf(" 9. Удалить элемент \n");
printf(" 10. Просмотр      \n");
printf(" 0. Выход          \n");
printf("Выберите действие: ");
}

```

```

int main()
{
    setlocale(LC_ALL, "Rus");

    int choice;

    do
    {
        show_menu();
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                priority_enqueue();
                break;
            case 2:
                priority_dequeue();
                break;
            case 3:
                view_priority_queue();
                break;
            case 4:
                search_by_priority();
                break;
            case 5:
                enqueue();
                break;
            case 6:
                dequeue();
                break;
            case 7:
                view_queue();
                break;
            case 8:
                push();
                break;
            case 9:
                pop();
                break;
            case 10:
                view_stack();
                break;
            case 0:
                printf("Выход из программы...\n");
                break;
            default:
                printf("Неверный выбор\n");
                break;
        }

        if (choice != 0) {

```

```
        printf("\nНажмите Enter для продолжения...");  
        getchar();  
        getchar();  
    }  
  
    } while (choice != 0);  
  
    printf("Очистка памяти...\n");  
    clear_all_lists();  
    printf("Программа завершена.\n");  
  
    return 0;  
}
```

Вывод

В ходе выполнения лабораторной работы были разработаны программы для выполнения заданий Лабораторной работы №3. В процессе выполнения работы были использованы знания о динамических списках.

Ссылка на удаленный репозиторий GitHub:

<https://github.com/MakarMn/LiOAvIZ>